

Guia de Oportunidades para Toda a Vida 89 — Desenvolvedor de Software

Versão: 2.0 — mestre de trabalho controlado

Idioma: Português do Brasil (pt-BR)

Referência principal dos EUA: O*NET-SOC 15-1252.00 — Software Developers

Comparação do Canadá: NOC 21232 — Software developers and programmers

Comparação da Colômbia: CUOC 25120 — Desarrolladores de software

Data de revisão: 2026-08-22

Fonte inglesa congelada: blob ef6a140a6dae98e48560a2de40365053bd914755

O que é esta carreira

Um desenvolvedor de software analisa necessidades, projeta e constrói software, testa e melhora soluções, documenta decisões e apoia o produto durante mudanças e operação. Pode trabalhar em aplicações desktop ou móveis, serviços cloud, componentes embarcados, sistemas internos, APIs, processamento de dados, plataformas ou utilitários especializados.

O título é amplo. Algumas pessoas desenvolvem funções de aplicações; outras trabalham em sistemas distribuídos, cloud, ferramentas DevOps, software embarcado, sistemas empresariais, serviços de dados ou infraestrutura. Ao longo da carreira, é mais importante compreender requisitos, escrever soluções sustentáveis, testar hipóteses, depurar falhas, proteger dados e aprender novas tecnologias do que memorizar muitos idiomas de programação.

Nos Estados Unidos há uma correspondência direta em **O*NET-SOC 15-1252.00 — Software Developers**, Bright Outlook e atualizada em 2026. O Canadá corresponde a **NOC 21232 — Software developers and programmers**. Na Colômbia há correspondência direta em **CUOC 25120 — Desarrolladores de software**.

Por que continua sendo uma carreira forte

Software sustenta finanças, logística, saúde, governo, comunicações, transporte, manufatura, cibersegurança, mídia e educação.

A projeção dos EUA mostra **16% de crescimento entre 2024 e 2034** e cerca de **115,200 aberturas anuais**, incluindo crescimento e reposição. Esses números descrevem a ocupação nacional e não garantem emprego individual.

Um desenvolvedor sólido combina:

- definição de problemas;
- fundamentos de programação;
- estruturas de dados e algoritmos;
- design de software;
- APIs e persistência;
- testes;
- debugging e observabilidade;
- segurança e privacidade;
- controle de versão;
- documentação;
- disciplina de release;
- uso responsável de automação e IA.

Famílias de funções

Desenvolvedor de aplicações

Constrói aplicações para usuários/processos de negócio e pode trabalhar em interface, serviços e dados.

Desenvolvedor back-end/serviços

Foca APIs, lógica de negócio, persistência, mensageria, cache, integração e confiabilidade.

Desenvolvedor de plataforma/infraestrutura

Constrói tooling, plataformas internas, automação e software próximo à infraestrutura.

Desenvolvedor móvel

Trabalha com apps nativos ou multiplataforma, permissões, armazenamento, rede e ciclo de vida do dispositivo.

Desenvolvedor de sistemas/embarcado

Trabalha mais próximo de sistemas operacionais, dispositivos, hardware, redes ou código sensível a desempenho.

Comece pelos requisitos

Antes de programar, esclareça:

1. Qual problema estamos resolvendo?
2. Qual comportamento é exigido?
3. O que está fora do escopo?
4. Quais inputs e outputs existem?

5. Quais requisitos de performance, disponibilidade, privacidade ou segurança se aplicam?
6. Quais sistemas/interfaces são afetados?
7. Qual evidência mostrará que a mudança funciona?
8. Quem aprova requisito e release?

Quando requisitos forem contraditórios, documente e escale. Não invente comportamento silenciosamente após implementar.

Design antes da implementação

Conforme o risco, avalie componente responsável, dados, contratos de API, falhas de dependências, compatibilidade, migration/rollback, controles de acesso, observabilidade e estratégia de testes.

Decisões arquiteturais podem pertencer a engenheiros seniores, arquitetos ou equipes de plataforma/segurança. Contribua com evidências sem assumir autoridade não atribuída.

Fundamentos de programação

Aprenda conceitos transferíveis:

- variáveis e tipos;
- controle de fluxo;
- funções/métodos;
- módulos/pacotes;
- coleções;
- erros/exceções;
- interfaces/abstrações;
- conceitos orientados a objetos e funcionais quando relevantes;
- comportamento assíncrono/concorrente;
- testes e debugging.

Aprenda um stack com profundidade suficiente para entender execução real antes de tentar dominar muitos idiomas.

Estruturas de dados e algoritmos

Entenda listas/arrays, mapas/dicionários, sets, stacks/queues, árvores/grafos em nível apropriado, busca/ordenação, iteração e complexidade de tempo/espço.

Nem toda vaga exige algoritmos avançados, mas compreender complexidade ajuda a evitar soluções ineficientes.

Tecnologias atuais

O*NET mostra para 2025:

- Python **29%**;
- AWS **26%**;
- Java **25%**;
- SQL **24%**;
- JavaScript **20%**;
- Azure **19%**;
- Kubernetes e Git **14%**;
- RESTful API, React e Docker **13%**;
- C# **12%**;
- C++ e Angular **10%**;
- CSS e Linux **9%**;
- Jenkins CI, HTML e TypeScript **8%**;
- Node.js, JIRA, GitHub e NoSQL **7%**;
- PostgreSQL, Terraform, Kafka e C **6%**;
- Spring Boot, Go e Spring Framework **5%**.

São sinais de vagas, não requisitos universais.

APIs e contratos

Interfaces podem ser REST/HTTP, RPC, mensageria/eventos, SDKs, schemas de banco ou formatos de arquivo. Defina inputs, outputs, erros, versionamento e compatibilidade. Não mude silenciosamente contratos usados por consumidores.

Bancos de dados e persistência

Conceitos úteis:

- tabelas/documentos;
- chaves/relacionamentos;
- índices;
- transações;
- consistência;
- migrations;
- conexões;
- cache;
- retenção;
- responsabilidades de backup/recovery.

Use consultas parametrizadas ou ORM aprovado. Não concatene input não confiável em SQL.

Autenticação e autorização

Autenticação estabelece identidade. Autorização determina permissões. Siga o modelo aprovado, privilégio mínimo e autorização no servidor. Ocultar UI não protege recursos.

Não enfraqueça autorização apenas para fazer uma funcionalidade passar em testes.

Git e code review

Use commits claros, branches conforme política, pull/merge requests, revisão por pares e resolução de conflitos.

Nunca faça commit de credenciais, API keys, tokens, certificados ou segredos. Se um segredo entrar no histórico, siga rotação/incidente; apagar uma linha visível pode não removê-lo do histórico.

Testes e QA

Podem incluir unitários, integração, contrato/API, componente, end-to-end, regressão, desempenho, acessibilidade e segurança dentro de autorização.

Uma suíte aprovada é evidência, não prova de software sem defeitos ou seguro.

Debugging e observabilidade

Fluxo útil:

1. reproduzir;
2. identificar versão/ambiente;
3. reduzir o problema;
4. revisar logs, métricas, traces, inputs e estado;
5. formular hipótese;
6. testá-la;
7. aplicar uma correção segura;
8. adicionar/ajustar testes;
9. retestar e monitorar.

Use logs estruturados, métricas, tracing quando aplicável, IDs de correlação, health checks e alertas. Não registre senhas, tokens, dados completos de pagamento ou informação pessoal desnecessária.

Erros e resiliência

Planeje input inválido, timeouts, falhas parciais, serviços indisponíveis, duplicatas, retries e degradação. Use retries limitados/backoff e idempotência quando apropriado. Retentar tudo cegamente pode piorar uma falha.

Concorrência e assincronia

Conforme o papel, entenda race conditions, sincronização, estado compartilhado, filas, ordenação, eventual consistency, cancelamento/timeouts e processamento duplicado.

Código assíncrono não é automaticamente correto só porque compila.

Dependências e supply chain

Use dependências necessárias e confiáveis, acompanhe advisories, versões e licenças quando necessário, teste upgrades e evite pacotes abandonados/código desconhecido.

Configuração e segredos

Proteja senhas de banco, API keys, credenciais cloud, OAuth secrets, signing/encryption keys e certificados. Use sistemas aprovados de secrets/configuration.

CI/CD

Um pipeline confiável identifica commit/versão, dependências, resultados de testes, artifacts, ambiente e approvals/gates. Não ignore gates obrigatórios apenas porque release manual seria mais rápida.

Deploy, mudança e rollback

Antes de produção, entenda versão, configuração, migrations, dependências, monitoramento, rollback/forward-fix, compatibilidade de dados e autoridade de release.

Acesso técnico não equivale a autoridade de produção.

Cloud e responsabilidade compartilhada

AWS, Azure, Docker e Kubernetes aparecem frequentemente. Serviços cloud não assumem automaticamente todas as responsabilidades de identidade, aplicação, configuração, segredos, dados e código. Siga o modelo do serviço e a arquitetura organizacional.

Desempenho

Meça antes de otimizar. Observe CPU/memória, I/O, rede, queries, cache, concorrência, pools de conexão, payloads e dependências. Load tests devem respeitar limites autorizados.

Desenvolvimento seguro

Práticas relevantes incluem validação, tratamento seguro de dados/output, autenticação/autorização, queries parametrizadas, defaults seguros, secrets, dependências, logging, code review e testes autorizados.

NIST SSDF e OWASP são referências úteis. Não dão permissão para atacar ou fazer pentest de sistemas.

Privacidade e acessibilidade

Use apenas dados necessários e siga regras de acesso, retenção, exclusão, dados de teste e logging. Não invente exigências legais; escale para a função responsável.

Conforme o produto, considere teclado, estruturas semânticas, labels, foco, contraste, erros compreensíveis e compatibilidade assistiva. Scanner automático não prova conformidade legal.

IA responsável

IA pode ajudar com explicação, scaffolding, refactoring, testes, dados sintéticos, documentação e debugging quando permitido.

Não envie código proprietário, dados de clientes, segredos, credenciais ou informação não publicada para ferramentas não aprovadas. Revise e teste código gerado. Verifique APIs/pacotes, segurança e licenças. Não permita deploy autônomo fora da governança. Output de IA não é evidência de execução.

Documentação e manutenção

Documente propósito, decisões, setup, configuração, APIs, modelos de dados, deployment, runbooks, limites, troubleshooting e ownership quando útil. Prefira mudanças pequenas e revisáveis a complexidade desnecessária.

Limites éticos e profissionais

Não se deve:

- fabricar resultados de testes/performance;
- esconder defeitos de alto impacto;
- implantar sem autoridade;

- contornar reviews/controles de segurança;
- confirmar segredos;
- usar dados de produção em demos pessoais;
- enfraquecer autorização sem requisitos aprovados;
- explorar sistemas fora de permissão;
- afirmar segurança/ausência de bugs só porque testes passaram;
- publicar código ou arquitetura privada do empregador.

Educação e entrada — Estados Unidos

O*NET coloca a ocupação em **Job Zone Four — Considerable Preparation Needed**. Respostas atuais para novas contratações: aproximadamente **85% bachelor's degree**, **5% associate degree** e **5% master's degree**. Não são regras universais.

CareerOneStop/American Job Centers ajudam a localizar WIOA e outras rotas. Elegibilidade e financiamento variam.

O*NET lista títulos de Registered Apprenticeship como **Application Developer**, **Commercial Drone Software Developer**, **Devops Engineer (Nof)** e **Software Developer (Nof)**. Verifique vagas reais em [Apprenticeship.gov](https://www.apprenticeship.gov).

Canadá

Job Bank usa **NOC 21232 — Software developers and programmers**. Geralmente exige bachelor's degree em computação/software engineering ou programa com programação significativa **ou** college program relacionado.

Job Bank identifica atualmente a ocupação como **não regulamentada no Canadá**.

Salários Canadá

- **C\$30.00/hora** baixo;
- **C\$48.08/hora** mediano;
- **C\$76.92/hora** alto.

Perspectiva Canadá

Demanda e oferta nacionais 2024-2033 devem ficar amplamente equilibradas. Perspectivas provinciais de três anos variam; confira a localização.

Colômbia

CUOC 25120 — Desarrolladores de software é correspondência direta, nível de competência 4, cobrindo análise, design, desenvolvimento, testes, manutenção e implementação.

Não se fabrica salário nacional representativo a partir de indicadores históricos/não representativos.

SENA

Análisis y desarrollo de software

- Tecnólogo;
- **3.984 horas**;
- formação titulada;
- requisitos, análise, design, desenvolvimento, implementação e qualidade;
- verifique turma, modalidade, vagas e admissão.

América Latina e Caribe

OIT/Cinterfor ajuda a localizar instituições nacionais de formação profissional. Não garante cursos, bolsas, vagas ou financiamento.

Salários e perspectiva

Estados Unidos oficial

Percentil	Anual	Por hora
10	\$82,460	\$39.64
25	\$105,210	\$50.58
Mediana	\$135,980	\$65.38
75	\$171,980	\$82.68
90	\$214,670	\$103.21

2024-2034:

- emprego 2024: **1,693,800**;
- projetado 2034: **1,961,400**;
- crescimento: **16%**;
- aberturas anuais: **115,200**.

Contexto não governamental adjacente

A URL da Indeed para Software Developer redireciona atualmente para **Software Engineer**. A página, atualizada em **10 de agosto de 2026**, informa aproximadamente:

- média **\$135,356/ano**;
- baixa **\$80,008/ano**;
- alta **\$228,992/ano**;
- **39.3 mil** observações;
- **36 meses** de vagas;
- bônus em dinheiro **\$5,000/ano**.

Por causa da redireção, esses valores são apenas contexto adjacente, não estatística exata do título Software Developer nem fonte oficial.

Sequência prática

1. Fundamentos: um idioma, Git, debugging, testes e estruturas básicas.
2. Aplicações/dados: API ou interface, persistência, validação e erros.
3. Engenharia: review, CI, logging, configuração/secrets e dependências.
4. Produção: deployment, rollback, observabilidade, cloud, segurança e performance.
5. Especialização: aplicações, back end, mobile, plataforma, cloud, embarcado, dados ou outro domínio.

Portfólio seguro

Use software próprio, open source, licenciado ou demo e dados sintéticos/públicos. Mostre requisitos, decisões de design, histórico Git, APIs/dados, testes, configuração segura, CI, documentação e deployment demo controlado.

Não publique código do empregador, dados de clientes, credenciais, infraestrutura privada, endpoints internos ou vulnerabilidades sem autorização.

Plano de quatro semanas

Semana 1

Escolha um idioma, crie programas pequenos e use Git e unit tests.

Semana 2

Construa aplicação/serviço com API/interface, persistência, validação e tratamento de erros.

Semana 3

Adicione integration tests, logging estruturado, configuração de ambientes e pipeline CI simples.

Semana 4

Documente arquitetura, premissas de segurança e limites; crie release demo controlada e bullets de currículo precisos.

Títulos de busca

- Software Developer;
- Junior Software Developer;
- Software Engineer;
- Application Developer;
- Back-End Developer;
- Platform Developer;
- Cloud Developer;
- Systems Developer;
- Integration Developer;
- Java Developer;
- Python Developer;
- .NET Developer;
- Mobile Developer;
- DevOps Engineer quando desenvolvimento for central.

Perguntas antes de aceitar uma vaga

- Que software a equipe possui?
- Quais linguagens/frameworks realmente usa?
- Como requisitos e arquitetura são decididos?
- Como funcionam code review e testes?
- Como releases são aprovadas/revertidas?
- Quem atende incidentes/on-call?
- Como segredos e dependências são gerenciados?
- Quais responsabilidades de segurança/privacidade pertencem aos desenvolvedores?
- Como dívida técnica é priorizada?
- O que diferencia júnior de sênior?

Fontes e links de verificação

Estados Unidos

- O*NET details: [Source link](#)
- O*NET summary: [Source link](#)
- O*NET Job Zone: [Source link](#)
- O*NET wages: [Source link](#)
- O*NET outlook: [Source link](#)
- O*NET technologies: [Source link](#)

- CareerOneStop WIOA: [Source link](#)
- Apprenticeship.gov: <https://www.apprenticeship.gov/>
- Indeed contexto adjacente: [Source link](#)

Canadá

- Job Bank summary: [Source link](#)
- Job Bank requirements: [Source link](#)
- Job Bank wages: [Source link](#)
- Job Bank outlook: [Source link](#)
- Canada training: [Source link](#)

Colômbia e América Latina

- CUOC 25120: [Source link](#)
- SENA Análisis y desarrollo de software: [Source link](#)
- OIT/Cinterfor: <https://www.oitcinterfor.org/statsfp/paises>

Segurança, IA e acessibilidade

- NIST SSDF: <https://csrc.nist.gov/pubs/sp/800/218/final>
- OWASP WSTG: [Source link](#)
- NIST AI RMF: [Source link](#)
- Section 508: <https://www.section508.gov/create/>
- WCAG 2.2: <https://www.w3.org/TR/WCAG22/>

Aviso importante

Este guia fornece informação geral educacional e de carreira. Não garante emprego, renda, admissão, financiamento, apprenticeship, certificação, promoção, segurança, acessibilidade ou outro resultado.

Não se declara certificação humana independente, acreditação profissional, revisão jurídica, avaliação de segurança, certificação de acessibilidade, certificação cloud/vendor ou tradução certificada salvo documentação separada.

Autor e assistência de IA

Criado e dirigido por **Alberto “Al” Leiva**. O ChatGPT apoiou pesquisa, organização, edição, tradução e preparação documental sob a direção do autor. O autor permanece responsável pelas decisões editoriais e de publicação.

Licença

Salvo indicação contrária, este material é licenciado sob **CC BY-NC-SA 4.0**.